Driving by Itself

Pedro Pathing 3 for FIRST Tech Challenge

Where am I, and how do I get there?

Version 2 | Built from pedropathing.com/docs

Read the Programming 101 deck first.

HOW TO USE THIS DECK

Two stops. Then you can read our drive code.

1. Knowing where you are — localizer, follower, Pose
2. Making it move — paths, TeleOp, tuning

Warning, and this one will save you hours. Pedro 3 is described by its own authors as a **major update and full rewrite**. Tutorials, videos and code written for Pedro 2 will **not** match. If a guide does not say Pedro 3, close it.

This deck assumes you have read **Programming 101**: objects, methods, `init()` and `loop()`.

Read more: Pedro → What's new with Pedro 3 · pedropathing.com/docs

1

Knowing Where You Are

Before a robot can drive itself anywhere, it has to know where it is standing and which way it is facing.

IN THIS PART

1. Pedro answers two questions
2. Pose: x, y and heading
3. Building the follower
4. Why update() matters

Eight words you are about to meet.

localizer	the part that answers “where am I?”
follower	the part that answers “how do I get there?”
Pose	x, y and heading — a spot plus a facing
heading	which way the front of the robot points

Constants	the file holding your robot’s measurements
`follower.update()`	the call that must run every loop
`setPose()`	tell Pedro where the robot is starting
odometry	measuring movement from wheel rotation

Read more: Pedro → Setting Up Your Autonomous

PEDRO — WHAT IT IS FOR

Pedro answers two questions, over and over.

Localizer

“Where am I?”

reads odometry / encoders

→

Follower

“How do I get there?”

sets the drive motor powers

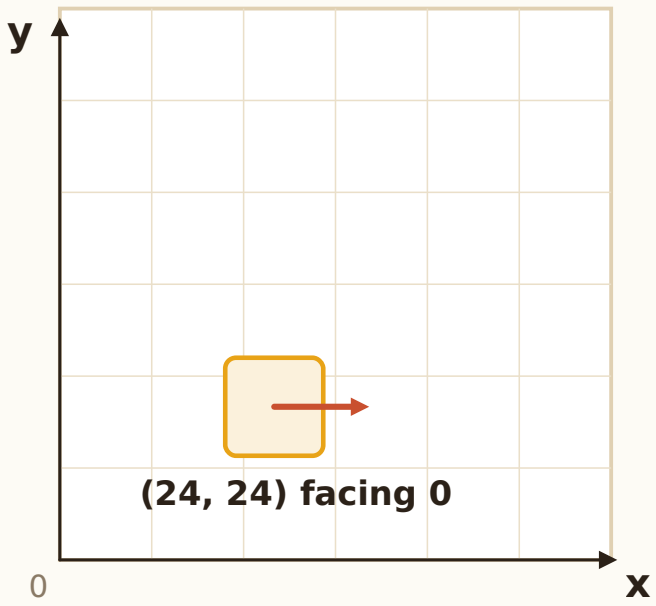
Together they are your robot’s GPS plus its driver.

Think of a taxi. The satnav knows where the taxi is. The driver knows how to work the steering wheel and pedals to get it somewhere else. Pedro is both of those.

Both halves run inside your `loop()`. Same fifty-times-a-second heartbeat from Programming 101 — Pedro is just one more thing that has to happen on every pass.

Read more: Pedro → Setting Up Your Autonomous

A Pose is where you stand plus which way you face.



x	how far across the field
y	how far up the field
heading	which way the front is pointing

All three together = one Pose

```
PoseFactory p = PoseFactory.degrees();
Pose startPose = p.of(24, 24, 0);
```

Watch the units: you **type** degrees with `PoseFactory.degrees()`, but `follower.pose()` hands back **inches and radians**. Pedro also uses **its own** field coordinates, not the official FTC ones.

Read more: Pedro → Creating a Pose · Pedro → Coordinates

Build it in `init()`. Feed it in `loop()`. Every loop.

```java
private Follower follower;

@Override
public void init() {
    follower = Constants.create(hardwareMap);
    follower.setPose(startPose);
}

@Override
public void loop() {
    follower.update();
}
```

> **`follower.update()` must run every single loop.** Miss it and Pedro stops knowing where the robot is. This is the number one reason Pedro “does not work” for new teams.

Look at where it sits: inside `loop()`. Straight back to the goldfish — read, decide, act, fifty times a second.

> And now the golden rule has teeth: a blocking `while` loop would starve `follower.update()` and the robot would lose track of itself.

Read more: Pedro → Setting Up Your Autonomous · Pedro → TeleOp Usage

Eight words you now own.

localizer	the part that answers “where am I?”
follower	the part that answers “how do I get there?”
Pose	x, y and heading — a spot plus a facing
heading	which way the front of the robot points
Constants	the file holding your robot’s measurements
`follower.update()`	the call that must run every loop
`setPose()`	tell Pedro where the robot is starting
odometry	measuring movement from wheel rotation

Read more: Pedro → Creating a Pose

2

Making It Move

Now that Pedro knows where the robot is, we can tell it where to go — and teach it how our particular robot behaves.

IN THIS PART

1. Paths: line and curve
2. Driving in TeleOp
3. Tuning is measuring
4. Where to get help

Ten words you are about to meet.

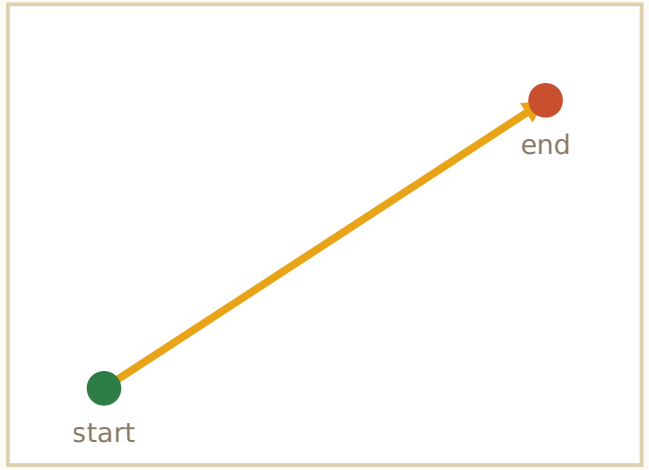
path	a route from one Pose to another
`line()`	a straight path
`curve()`	a bent path
control pose	the point that bends a curve
interpolation	how the heading changes along the way

`follower.manual()`	drive from the gamepad sticks
robot-centric	forward means “the way the robot faces”
field-centric	forward means “away from the driver”
AutoTune	the web page for measuring your robot
Foresight	Pedro’s path-following algorithm

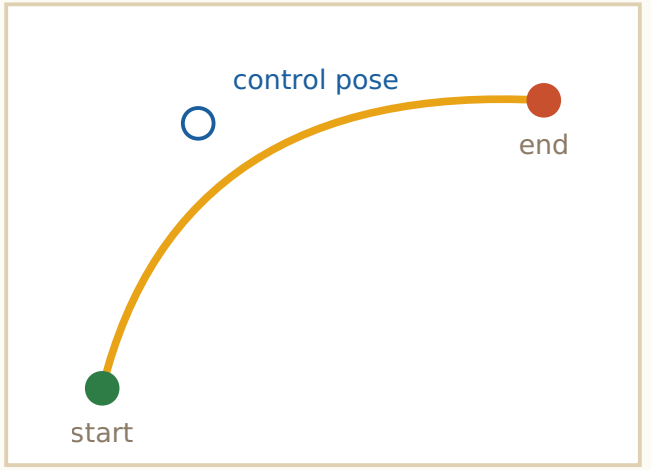
Read more: Pedro → Creating a Path

A path is a line or a curve between two Poses.

line()



curve()




```
line(startPose, park).linear(startPose, park);

curve(startPose, controlPose, park)
        .linear(startPose, park);
```

Add a **control pose** in the middle and the straight line bends towards it. Add more to bend it further.

Draw paths in a browser first: visualizer.pedropathing.com. Far cheaper than discovering the problem on carpet.

Read more: Pedro → Creating a Path · Pedro → Bezier Curves · Visualizer

Pedro still drives. You just hold the sticks.

```
@Override
public void loop() {
    follower.manual(
            -gamepad1.left_stick_y,
            gamepad1.left_stick_x,
            gamepad1.right_stick_x
    );

    follower.update();
}
```

Three numbers: **forward**, **sideways**, **turn**. Pedro works out what each of the four wheels should do.

Spot the minus sign on `left_stick_y`. Exactly the gamepad quirk from Programming 101, in real code.

This is **robot-centric**: forward means “the way the robot is facing”. For **field-centric** — forward means “away from the driver” — Pedro 3 provides `ManualDrive.fieldCentric(...)`.

Tuning is measuring your robot. Not guessing numbers.

Pedro cannot know how heavy our robot is, how fast it accelerates, or how far it slides when you let go. **Tuning is how we tell it.**

Pedro 3 does this through **AutoTune**, a web page hosted **on the robot itself**. You connect your laptop to the robot and open it in a browser.

THREE STEPS, STRICTLY IN THIS ORDER

1	**Drivetrain** — motor names and directions
2	**Localization** — teach it to know where it is
3	**Foresight** — the path-following algorithm

Then run the **Tests** procedure to check the whole thing actually works before trusting it in a match.

Read more: Pedro → Tuning · Pedro → Test · Pedro → Troubleshooting

You are not the first person to be stuck on this.

Where	Best for	Link
Pedro docs	the correct, current way to do anything	`pedropathing.com/docs`
Visualizer	drawing and testing paths in a browser	`visualizer.pedropathing.com`
Troubleshooting page	“it compiles but the robot does something silly”	`pedropathing.com/docs/pathing/faq`
Pedro Discord	asking a human when the docs have run out	linked from `pedropathing.com`
Javadoc	what exactly does this method do	`javadoc.io/doc/com.pedropathing`

Read more: When you ask for help, bring evidence: what you expected, what happened, your constants, and a telemetry screenshot. See gm0 → How to Ask Effective Questions

Ten words you now own.

path	a route from one Pose to another
`line()`	a straight path
`curve()`	a bent path
control pose	the point that bends a curve
interpolation	how the heading changes along the way

`follower.manual()`	drive from the gamepad sticks
robot-centric	forward means “the way the robot faces”
field-centric	forward means “away from the driver”
AutoTune	the web page for measuring your robot
Foresight	Pedro’s path-following algorithm

Read more: Pedro → TeleOp Usage

CHEAT SHEET

Five sentences. Photograph this slide.

1. Pedro answers two questions: **where am I** (localizer) and **how do I get there** (follower).
2. A **Pose** is x, y and heading — a spot on the field plus a facing.
3. Build the follower in `init()`. Call `follower.update()` every single `loop()`.
4. A **path** is a line or a curve between two Poses. Draw it in the visualizer first.
5. **Tuning is measuring**, in order: drivetrain, then localization, then Foresight.

> **And the one warning:** Pedro 3 is a full rewrite. Anything written for Pedro 2 will not match. Always check the version.

Read more: Every slide in this deck cites its source at the bottom. All of it comes from pedropathing.com/docs.